

Formation FPGA

Projet SoC Tracking

Plan de validation

Introduction

Les spécifications attendues pour notre produit *SoC Tracking* sont détaillées dans le document **Software Requirements Specifications (SRS)**.

Nous choisissons de classer ces spécifications en quatre catégories, en fonction de l'ordre que nous prévoyons pour les implanter et donc les valider :

- Les entrées/sorties, qui demandent :
 - soit de créer un signal vidéo sous la forme AXI4-Stream ([PL-IP-005]), en mettant en mémoire ([PS-001]) puis en lisant une source externe ([PL-IP-000]), ou en générant des images selon un schéma prédéfini ([PL-IP-001]) ;
 - soit de transformer un signal numérique vidéo sous la forme AXI4-Stream en un signal VGA analogique pouvant être afficher sur un écran compatible([PL-DISP-002]).
- Les étapes de détection de contours, qui demandent de créer un kernel depuis un flux vidéo AXI4-S ([PL-IP-004]), d'appliquer un filtre Gaussien ([PL-IP-006]) et un filtre Harris ([PL-IP-007]), d'effectuer une binarisation ([PL-IP-008]) et d'afficher le résultat au-dessus de l'image d'origine ([PL-DISP-001]).
- Les spécifications de paramétrisation qui viennent préciser le fonctionnement des entités précédentes, à l'initialisation ou à chaud : sélection de la source vidéo ([PL-IP-002] et [PL-IP-003]), paramétrisation du seuil de binarisation ([PL-IP-009]), choix de l'affichage de l'overlay ([PL-DISP-003]), mise en mémoire automatique de l'image dans la DDR ([PS-002]).
- Les spécifications de retours, qui donnent une information sur le traitement en cours : ici le retour de données sur le filtre Harris ([PL-IP-010] et [PS-003]).

Si les deux premières catégories peuvent être indépendantes pendant leur conception, il sera bien plus aisé d'effectuer des tests et des démonstrations sur le fonctionnement du filtre si un signal d'entrée fonctionnel existe et si il y a un moyen de visualiser le résultat sur un écran.

Les deux catégories suivantes, elles, nécessitent que les blocs que nous paramétrons soit fonctionnels. Comme elles impliquent la partie PS de notre SoC et des signaux AXI4-Lite compliqués à simuler, nous préférons les valider avec des tests/démonstrations sur carte, ce qui demande que les autres blocs fonctionnels autour soit déjà validés et fiables en fonctionnement réel.

Entrées/sorties

Cette première partie se concentre sur la validation des fonctionnalités de génération de signal vidéo AXI4-Stream, ainsi que leur affichage sur un écran.

[PL-IP-005] Standards de transferts [1]

Cette première spécification n'a pas vraiment à être validée en soit, puisqu'elle guide simplement la conception de notre architecture en conditionnant le format de nos flux d'images en entrée et en sortie des filtres, qui devront être au format AXI4-Stream.

De manière générale, nous suivrons cette spécification en utilisant le format AXI4-Stream pour l'ensemble de nos flux vidéo à l'intérieur du FPGA, de leur génération jusqu'à leur conversion de sortie au format VGA.

[PL-IP-001] Source Vidéo TPG [1]

La première fonctionnalité que nous allons implémenter, et donc valider, est la génération d'une source vidéo, en AXI4-Stream, en suivant un schéma de génération prédéfini (Test Pattern Generator, TPG) correspondant à la spécification (mire avec au moins un élément dynamique).

Nous abordons au départ cette source vidéo car elle ne dépend que des ressources FPGA PL, et non pas de la partie PS comme c'est le cas pour le DMA (le processeur est nécessaire pour mettre en mémoire l'image dans la DDR).

Deux pistes distinctes sont envisagées pour la réalisation de cette fonctionnalité :

- l'IP constructeur Xilinx « Video Test Pattern Generator », listée dans le catalogue d'IP de Vivado,
- le code VHDL fournis par Digilent dans les ressources « Cora-Z7-10-Pmod-VGA-2018.2-1 » fournies afin de faire fonctionner l'adaptateur Pmod VGA de Digilent pour carte

Cora Z7, qui contient une partie générant les signaux RGB selon un schéma de test prédéfini qui correspond à notre spécification.

La **validation** de cette spécification se fera en trois parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation, en créant sur Vivado un testbench où la sortie de notre TPG sera enregistré sur un moniteur RGB qui enregistrera chacun des canaux de couleur et permettra de reconstituer l'image de sortie.
- Démonstration : une fois la fonctionnalité d'affichage VGA également implémentée et simulée (voire fonctionnalité suivante), les deux blocs pourront être démontrés conjointement, avec affichage sur écran de notre TPG.

[PL-DISP-002] Affichage écran VGA [1]

Après avoir créé un point de départ pour notre AXI4-Stream notre architecture, la seconde fonctionnalité que nous allons implémenter est son point d'arrivée : la transformation de ce flux AXI4-Stream en un signal VGA analogique de dimension 640*480 et de fréquence 60 Hz, prêt à être afficher sur un écran compatible.

Une partie de la solution est déjà connue : nous allons utiliser l'adaptateur Pmod VGA de Digilent pour carte Cora Z7, qui permet de passer d'un signal VGA numérique (4 bits par canal RGB) sur les connecteurs Pmod de la carte à un signal VGA analogique sur port VGA spécifique.

Il nous reste à implémenter la conversion du flux AXI4-Stream en flux VGA numérique. Pour cela nous pourrons encore une fois nous inspirer du code VHDL dans les ressources « Cora-Z7-10-Pmod-VGA-2018.2-1 », qui effectue le cadencage des canaux RGB et la génération des signaux de synchronisation horizontaux et verticaux pour de l'affichage VGA numérique.

La **validation** de cette spécification se fera en trois parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation, en créant sur Vivado un testbench où notre bloc TPG (déjà simulé individuellement) sera relié à notre nouveau bloc d'affichage VGA. Le résultat de cette simulation sera comparé à celui obtenu pour le projet « Cora-Z7-10-Pmod-VGA-2018.2-1 ».
- Démonstration : les deux blocs, TPG et contrôleur d'affichage VGA pourront être démontrés conjointement. On doit voir apparaître le TPG sur notre écran.

[PL-IP-000] Source Vidéo mémoire [1] et [PS-001] Mise en mémoire manuelle pattern [1]

Nous allons ensuite implémenter notre deuxième origine possible pour notre flux AXI4-Stream : la lecture DMA d'une image stockée dans la mémoire DDR du SoC. Cette lecture nécessite naturellement que l'image ait été mise en mémoire, et inversement la lecture sert de preuve à la bonne mise en mémoire de l'image : les deux spécifications sont validées ensembles.

Une partie de la solution nous a été fournie, sous la forme du projet Vitis HLS « DMA_RGB24b », qui comprend notamment une IP pouvant être directement instanciée dans Vivado. Il nous reste à ajouter un bloc de contrôle de cette DMA, qui indique l'adresse de l'image dans la DDR et déclenche la lecture. Ces informations pourront être gérées par un registre, piloté par la partie PS via une connexion AXI4-Lite.

La **validation** de ces spécifications se fera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Démonstration : on construit une architecture comportant la partie PS, le lecteur DMA et son contrôleur, ainsi que le contrôleur d'affichage VGA. On teste cette architecture via Vitis, en utilisant la console XSCT pour charger la mémoire DDR et configuré les registres du contrôleur VGA. On doit voir notre image apparaître sur l'écran.

Détection de contours

Cette seconde partie va se concentrer sur la détection de contours, c'est à dire l'ensemble du processus permettant de passer d'une image couleur à l'overlay affichant de manière superposée à cette image ses points de contour avec une couleur vive.

Les différentes étapes de ce processus manipulant des flux d'image au format AXI4-Stream mais aux caractéristiques variées (pixel RGB 24 bits ou nuances de gris 8 bits, fenêtre de 9 pixels en 72 bits, dimensions d'image variées), il sera compliqué de faire des démonstrations individuelles de nos composants. La plupart seront uniquement simulés, individuellement ou par petits groupes, avant que la chaîne ne soit suffisamment longue pour obtenir un signal AXI4-Stream susceptible d'être converti en VGA pour une démonstration d'affichage sur écran.

[PL-IP-004] Architecture filtre à convolution [1]

Nos filtres travaillent sur des images en nuance de gris, alors que nous avons des sources de vidéo en RGB. La toute première étape est donc de réaliser un bloc qui transforme le flux AXI4-Stream RGB en flux (3 canaux de 8 bits, 24 bits au total) en nuance de gris (sur 8 bits).

La **validation** de ce bloc débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation : on construit un testbench, où un driver lit les différents canaux d'une image RGB pour alimenter notre bloc, dont la sortie est ensuite fournie à un moniteur qui enregistre l'image produite. Cette image doit être identique à celle que produit ImageJ quand on lui demande de convertir en niveaux de gris l'image originale.

La deuxième étape est de réaliser un bloc qui peut stocker un buffer de trois lignes à la fois de l'image en cours, pour en extraire une fenêtre glissante de trois pixels par trois, le kernel permettant ultérieurement d'appliquer un filtre.

La **validation** individuelle de ce bloc débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation : on construit un testbench, où un driver lit plusieurs fois une image en niveaux de gris pour alimenter notre bloc, dont la sortie est ensuite fournie à un moniteur qui enregistre le flux produit (en convertissant en entier relatif la valeur des 72 bits du flux de sortie, et le stockant comme un unique pixel). La suite de valeurs enregistrée en sortie n'a pas de sens en tant qu'image, mais elle permet de comparer les sorties successives qui doivent être identiques (il n'y a pas de décalage, le bloc produit bien pour chaque image un nombre de fenêtre correspondant à deux lignes et deux colonnes de moins que l'image d'origine).

La validation de ce bloc continue avec la spécification suivante.

[PL-IP-006] Filtre gaussien [1] et [PL-IP-007] Filtre Harris [1]

L'étape suivante est de réaliser le filtre Gaussien et le filtre Harris, qui appliquent chacun leurs coefficients et leur diviseur au kernel fourni.

La **validation** de chaque bloc de filtre débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).

- Simulation : on construit un testbench, où un driver lit plusieurs fois une image en niveaux de gris pour alimenter notre bloc buffer, dont la sortie de kernels est reçue par le filtre qui à son tour transmet son flux de sortie à un moniteur qui enregistre l'image produite. L'image produite doit être identique au résultat que donne ImageJ pour une convolution de l'image d'origine par la même matrice de coefficient. On compare aussi les images de sorties successives qui doivent être identiques.

[PL-IP-008] Filtre Harris binarisation [1]

L'étape suivante est de réaliser un bloc qui effectue une binarisation du flux d'images AXI4-Stream, en fonction d'un seuil paramétrable à l'instanciation du bloc.

La **validation** du bloc de binarisation débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation : on construit un testbench, où un driver lit une image en niveaux de gris pour alimenter notre bloc de binarisation, dont la sortie est transmise à un moniteur qui enregistre l'image produite. L'image produite doit être identique au résultat que donne ImageJ pour une binarisation de l'image d'origine par le même seuil.

En complément de ce bloc, on va élaborer un bloc de « zéro-padding », dont la fonction est de remplir les colonnes et lignes extérieures d'une image de zéros, pour augmenter les dimensions de l'image. Cela permettra de contrer l'effet des fenêtres glissantes, qui suppriment deux lignes et deux colonnes à chaque application, et retourner à la dimension originale de l'image de 640*480.

La **validation** du bloc de zéro-padding débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation : on construit un testbench, où un driver lit une image en niveaux de gris pour alimenter notre bloc zéro-padding, dont la sortie est transmise à un moniteur qui enregistre l'image produite. L'image produite doit contenir l'image de départ, entourée de pixels noirs.

En ajoutant un bloc qui transforme un flux AXI4-Stream en niveau de gris en flux AXI4-Stream RGB (chaque canal prend la valeur de niveau de gris), on a tous les composants pour appliquer la détection de contour et revenir à un flux affichable en VGA.

On peut donc mettre en place une première **validation** groupée de nos composants de détection de contour par démonstration, en faisant fonctionner une architecture :

TPG → RGB vers gris → Buffer/kernel → Filtre gaussien → Buffer/kernel → Filtre Harris → Zero-padding → Gris vers RGB → affichage VGA

(Le bloc de binarisation n'est pas inclus, des valeurs uniquement de '0' et '1' serait indistinguable en VGA.

[PL-DISP-001] Overlay [1]

Avant d'entamer la réalisation de l'overlay, il nous faut une copie du flux d'image AXI4-Stream RGB avant traitements, synchronisée avec le flux résultat des traitements. Pour cela, il nous faut un bloc capable de dupliquer un flux AXI4-Stream : le flux d'entrée est stocké dans le bloc, une sortie le restitue immédiatement (et reste synchronisée sur l'entrée) alors que l'autre peut le restituer avec un décalage.

La **validation** du bloc de duplication de flux débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation : on construit un testbench, où un driver lit une image en niveaux de gris pour alimenter notre bloc de duplication, dont les sorties sont reliées à deux moniteurs qui enregistrent les images produites. Pour vérifier le décalage du deuxième flux de sortie, le moniteur correspondant aura une période de reset initial bien plus longue que le reste des composants de l'architecture. Les deux images produites, ainsi que l'image de départ, doivent être identiques.

On peut enfin attaquer le dernier bloc de la détection de contour : l'overlay qui superpose la détection de contour à l'image d'origine. Un trio de paramètres RGB, choisis à l'instanciation du bloc, détermine la couleur sous laquelle apparaissent les pixels à '1' dans le flux binarisé, les autres pixels apparaissent avec leur valeur dans l'image d'origine.

La **validation** du bloc d'overlay débutera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Simulation : on construit un testbench, où un driver lit une image RGB, et un autre lit une image binarisée, et les deux alimentent l'overlay, dont la sortie est reliée à un moniteur qui enregistre l'image produite.

On peut donc mettre en place une deuxième **validation** groupée de nos composants de détection de contour par démonstration, en faisant fonctionner une architecture :

TPG → Duplication → RGB vers gris → Buffer/kernel → Filtre gaussien → Buffer/kernel → Filtre Harris → Zero-padding → Binarisation → Overlay → Affichage VGA

Paramétrisation

Dans cette troisième partie, on va partir de l'architecture validée finale de la partie précédente, mettre en place le composant PS et apporter des ajustements, en ajoutant à chaque fois un unique bloc, ou en en modifiant un déjà existant.

Ces modifications sont la plupart du temps assez légère, et consiste souvent à remplacer un paramètre qui était jusqu'à là défini à l'instanciation du bloc par un paramètre inscrit dans un registre, gouverné par AXI4-Lite par le PS, avec une définition fixée soit à l'initialisation du firmware, soit modifiable à chaud. Le PS et ses communications AXI4-Lite étant compliqués à simuler, et chaque modification touchant un aspect restreint, on privilégiera les validations par test ou démonstration.

[PL-IP-002] Sélection de source vidéo init [1]

Pour sélectionner la source vidéo à l'initialisation, on conçoit un bloc qui dispose de deux entrées et une sortie AXI4-Stream, ainsi qu'un Slave AXI4-Lite piloté par le PS. Le bloc aura deux bits de registre : le premier indique la source (TPG ou DMA) qu'on retrouve en sortie, le second indique que l'initialisation est faite, verrouillant la source et déclenchant le flux. Après verrouillage, modifier le premier bit de registre n'a aucun effet.

La **validation** de cette spécification se fera en deux parties :

- Inspection sur le résultat de notre conception (RTL et VHDL).
- Test : on reprend l'architecture en ajoutant la partie PS et le bloc sélecteur de source vidéo. On teste cette architecture via Vitis, en utilisant la console XSCT pour d'abord définir la valeur du registre choisissant la source, puis celle qui confirme la source et démarre le flux. On doit voir sur l'écran apparaître l'image ou la vidéo correspondant à notre choix, avec l'overlay de détection de contours.

[PL-IP-003] Sélection de source vidéo à chaud [2]

On modifie le bloc défini à l'étape précédente, pour qu'un changement du premier bit de registre puisse toujours effectuer un changement de source, mais que celui-ci ne se produise qu'au moment du premier pixel d'une image (signal AXI4-Stream TUSER). Le flux non-utilisé restant figé, il attend toujours à un premier pixel : soit parce que le flux n'a pas encore été utilisé depuis l'initialisation, soit parce qu'il s'est arrêté sur ce pixel au dernier changement de source. Le second bit du registre n'a plus d'effet de verrou, simplement de démarrage global du stream.

La **validation** de cette spécification se fera en deux parties :

- Inspection sur le résultat de notre conception (VHDL).
- Test : on reprend l'architecture précédente avec le bloc sélecteur de source vidéo modifié. On teste cette architecture via Vitis, en utilisant la console XSCT pour d'abord démarrer le flux, puis changer successivement de source. À chaque commande de changement de source, l'écran doit afficher l'image ou la vidéo correspondante, avec son overlay.

[PL-IP-009] Filtre Harris seuillage paramétrable [1]

On modifie le bloc de binarisation pour lui ajouter un Slave AXI4-Lite piloté par le PS. Le bloc possède un registre, où le PS indique la valeur du seuil de binarisation. Le calcul dans le bloc utilise la valeur du registre, au lieu d'un paramètre défini à l'instanciation du bloc.

La **validation** de cette spécification se fera en deux parties :

- Inspection sur le résultat de notre conception (VHDL).
- Test : on reprend l'architecture précédente avec le bloc de binarisation modifié. On teste cette architecture via Vitis, en utilisant la console XSCT pour enregistrer dans le registre différentes valeurs de seuil. À chaque valeur de seuil, l'écran doit afficher l'overlay mis à jour : plus le seuil est bas, plus il y a de pixels mis en avant sur l'overlay.

[PL-DISP-003] Mode d'affichage [2]

On modifie le bloc d'overlay pour lui ajouter un Slave AXI4-Lite piloté par le PS. Le bloc possède un registre d'un bit, où le PS indique le mode d'affichage. En fonction du mode d'affichage, le bloc laisse sortir l'image superposée de son overlay, ou bien seulement l'image. Pour éviter de créer un décalage entre les deux sources du bloc overlay, on continue de synchroniser l'entrée des deux flux dans le bloc, même si l'overlay n'est pas utilisé en sortie.

La **validation** de cette spécification se fera en deux parties :

- Inspection sur le résultat de notre conception (VHDL).
- Test : on reprend l'architecture précédente avec le bloc d'overlay modifié. On teste cette architecture via Vitis, en utilisant la console XSCT pour alterner dans le registre entre les deux modes d'affichage. À chaque changement de valeur, l'overlay doit s'ajouter ou disparaître de l'image sur l'écran.

[PS-002] Mise en mémoire automatique pattern [2]

Pas de modification de l'architecture FPGA ici. Les changements ont lieu côté firmware : à son initialisation, le programme détecte une éventuelle carte SD, lit son contenu, et l'inscrit dans la DDR à l'adresse de lecture de la DMA.

La **validation** de cette spécification se fera en deux parties :

- Inspection sur le résultat de notre conception (C).
- Test : on reprend l'architecture précédente. On teste cette architecture via Vitis, en utilisant la console XSCT pour passer à la source DMA, où en codant le firmware pour écrire à l'initialisation les registres choisissant la source DMA. L'image sur l'écran doit correspondre à celle inscrite sur la carte SD insérée dans la Cora Z7.

Spécifications de retours

[PL-IP-010] Filtre Harris retour de données [2] et [PS-003] Retour de données UART [2]

On modifie le fonctionnement du bloc de binarisation, pour qu'il calcule sur chaque image la valeur maximale de son entrée (le résultat du filtre Harris) et la position de ce maximum. On modifie le fonctionnement du registre AXI4-Lite Slave pour, en plus de recevoir depuis le PS la valeur du seuil de binarisation, permettre au PS de lire cette valeur maximal et les coordonnées correspondantes.

Côté firmware, on modifie le code pour régulièrement lire ces 3 valeurs de registre, et les envoyer sur le canal UART.

La **validation** de ces spécifications se fera en deux parties :

- Inspection sur le résultat de notre conception (VHDL et C).
- Test : on reprend l'architecture précédente, avec le bloc de binarisation modifié. On teste cette architecture via Vitis, et on met en place un terminal pouvant recevoir la communication UART de la carte. Le terminal doit recevoir de manière régulière la valeur maximale du filtre et les coordonnées correspondantes.